

Contents

GPT API에 보내는 프롬프트 모음 — 비개발자 안내	1
0. GPT를 두 가지 역할로 사용합니다	1
1. 생성자(LLM-A) 시스템 프롬프트	1
2. 생성자가 매 사이클 받는 본문 프롬프트 (실제 예시)	1
이 프롬프트가 무엇을 알려주는가	3
Task 부분의 4가지 연산자	3
3. 생성자가 답해야 하는 형식	3
4. 반성가(LLM-S) 시스템 프롬프트	3
5. 반성가가 매 사이클 받는 본문 프롬프트 (실제 예시)	4
6. 반성가가 답해야 하는 형식	4
7. 사용하는 OpenAI API 설정	5
8. 비용 추정 (gpt-4o-mini 기준)	5
9. 안전장치 — 잘못된 답이 와도 시스템이 안 망함	5
10. 한 줄 요약	5

GPT API에 보내는 프롬프트 모음 — 비개발자 안내

우리 시스템이 GPT-4o-mini에게 정확히 무엇을 묻는지, GPT가 무엇을 답해야 하는지를 그대로 보여주는 문서입니다.

0. GPT를 두 가지 역할로 사용합니다

역할	이름	무엇을 하는가
생성자	LLM-A	새로운 작업 우선순위 규칙(수식)을 짜냄
반성가	LLM-S	결과를 보고 “이게 왜 좋고 왜 나빴는지” 교훈 작성

같은 모델을 두 역할로 쓸 수도 있고, 다른 모델을 쓸 수도 있습니다 (예: 생성=강한 모델, 반성=싼 모델).

1. 생성자(LLM-A) 시스템 프롬프트

GPT에게 “너는 어떤 역할인지” 알려주는 짧은 안내입니다.

You are LLM-A, a heuristic-evolution agent. You output a SINGLE evalexpr expression as a dispatching priority rule. Higher score = higher priority. You MUST follow the Thought / Code output format exactly.

번역: > 너는 LLM-A, 휴리스틱 진화 에이전트야. 작업 우선순위 규칙을 evalexpr 한 줄 표현식으로 출력해. 높은 점수 = 높은 우선순위. Thought / Code 형식을 정확히 지켜야 해.

2. 생성자가 매 사이클 받는 본문 프롬프트 (실제 예시)

아래는 진짜로 GPT에게 보내는 prompt 그대로입니다 (S5 시나리오, crossover 연산 시):

You are evolving a dispatching rule for a dynamic Job Shop Scheduling problem under supply-chain disruption.

=== Scenario: S5 ===

Combined: part delay + material shortage + urgent surge + due shock + occasional breakdowns simultaneously.

=== Baseline Dispatching Rules ===

1. FIFO: ``0.0 - release``
2. EDD: ``0.0 - due``
3. SPT: ``0.0 - proc``
4. CR: ``0.0 - (due - now) / max_(remaining_proc, 0.001)``
5. Urgency: ``iff(urgent, 1.0, 0.0)``
6. WMDD: ``0.0 - max_(due, now + proc) / max_(penalty, 0.001)``
7. COVERT: ``(penalty / max_(proc, 0.001)) * max_(0.0, 1.0 - ...)``
8. ATC: ``(penalty / max_(proc, 0.001)) * exp_(0.0 - max_(0.0, ...) / ...)``
9. LPT: ``proc``
10. MWKR: ``remaining_proc``
11. LWKR: ``0.0 - remaining_proc``
12. MDD: ``0.0 - max_(due, now + proc)``

=== Variables (USE THESE EXACT NAMES – others will be rejected) ===

Job: release, due, slack, urgent, penalty,
 total_proc, remaining_proc,
 part_avail, time_to_avail, mat_risk, inbound_delay
Operation: proc, op_idx
Machine: machine_id, machine_queue, mach_util, mach_down
State: now, n_ready, n_running, n_jobs,
 supply_delay_level, urgent_ratio, disruption_level,
 avg_inbound_delay

Notes:

- ``urgent`` is 1 or 0 (NOT a float). Use it as: `iff(urgent, A, B)`.
- ``slack`` already equals `due - now`. Don't recompute.
- ``proc`` is the candidate operation's processing time. ``remaining_proc`` is the rest of the job's work after this op.

=== Helper functions (numeric in / numeric out) ===

`iff(cond, then, else)` - `cond≠0 → then, else else`
`gt(a, b), lt(a, b), eq(a, b)` - `return 1.0 / 0.0`
`max_(a, b), min_(a, b)`
`clamp(x, lo, hi)`
`exp_(x)` - natural exponential

DO NOT use: `math::*`, `exp()`, `max()`, `abs()`, Python operators (and/or/not),
if-else statements, multi-line code, function definitions, comments,
or any variable name not listed above.

=== Output format ===

Return ONLY two sections, no extra commentary, no markdown fences:

Thought: <one or two sentences explaining the heuristic idea>

Code: <a single evalexpr expression returning a float; higher = higher priority>

=== Memory (top relevant lessons) ===

[Failure | S5 | Δ=-10.0%] 긴급도 가중치 너무 크면 손해
 applies: S5 복합 충격에서 다른 변수와 혼합 시
 detail: 긴급도만 강조하면 일반 작업 지연 폭증

=== Previous Performance (elite, lower obj is better) ===

#1 obj=1230.0 makespan=900 mean_tard=180.0 feas=0.92 gap_vs_FIFO=-22.5%
 expr: `0.0 - max_(due, now + proc) / max_(penalty, 0.001)`
#2 obj=1320.0 makespan=950 mean_tard=195.0 feas=0.93 gap_vs_FIFO=-19.0%
 expr: `(penalty / max_(proc, 0.001)) * exp_(0.0 - slack / 3.0 / proc)`

=== Task ===

Combine the strengths of the top two elite rules into one new expression.

Constraints:

- The expression MUST be a single evalexpr expression returning a Float.
- Higher score = higher priority.
- Use only the variables and helper functions listed above.
- Feasibility ($\text{part_avail} \leq \text{now}$) is enforced by the engine; you may use `part_avail` and `time_to_avail` as features but you do not need to gate on them.

Return Thought + Code only.

이 프롬프트가 무엇을 알려주는가

섹션	무엇을 알려주는가
Scenario	어떤 외부 충격 상황인지 (S0~S5)
Baseline Dispatching Rules	12개 고전 규칙 — 비교/조합 재료
Variables	사용 가능한 변수 30종 (작업/기계/상태 정보)
Helper functions	if/max/min/exp 등 6가지 함수
금지 사항	Python 문법, 멀티라인, 코멘트 등은 못 씀
Output format	“Thought:” + “Code:” 두 줄만
Memory	직전 사이클에서 얻은 교훈
Previous Performance	가장 잘했던 규칙 + 성능 수치
Task	이번엔 어떤 연산자(생성/교차/수정/단순화)를 쓸지

Task 부분의 4가지 연산자

매 사이클에 GPT는 4가지 다른 작업을 요청받습니다:

연산자	한국어	무엇을 하라는 지시
explore (E1)	탐색	기존 규칙과 구조가 다른 새 규칙 생성
crossover (E2)	교차	상위 2개 규칙의 장점을 합쳐 새 규칙
modify (M1)	수정	상위 규칙의 가중치/임계값 조정
simplify (M2)	단순화	상위 규칙에서 불필요한 항 제거

EOH (Liu 2024) 논문의 4 연산자를 그대로 사용.

3. 생성자가 답해야 하는 형식

GPT는 정확히 이런 형식으로 답해야 합니다:

Thought: ATC의 지수 형태를 살리되 `mat_risk`를 직접 가중치로 결합해
자재 부족 위험이 높은 작업이 더 빨리 처리되도록 설계.

Code: $(\text{penalty} / \max(\text{proc}, 0.001)) * \exp(0.0 - \text{slack} / (3.0 * \text{proc}))$
+ $5.0 * \text{mat_risk}$

시스템은 Code: 뒤에 오는 한 줄을 추출해서 시뮬레이터에 넘깁니다.

4. 반성가(LLM-S) 시스템 프롬프트

You are LLM-S, a reflector. You output zero or more LESSON: ... END blocks.
No prose outside those blocks.

번역: > 너는 LLM-S, 반성가야. LESSON: ... END 블록을 0개 이상 출력해. 다른 글은 쓰지 마.

5. 반성가가 매 사이클 받는 본문 프롬프트 (실제 예시)

You are LLM-S, the reflector in an EvoDR-style dual-expert evolution loop for scheduling-rule discovery under scenario S5
(Combined: part delay + material shortage + urgent surge + due shock + ...).

The lists below are PRE-CLASSIFIED. Each rule was scored against the best fixed baseline (FIFO/EDD/SPT/CR/Urgency/WMDD/COVERT/ATC) under the scenario-specific weights of **창종설 §6-2**. Rules $\geq 5\%$ better are labelled SUCCESS; rules $\geq 5\%$ worse are labelled FAILURE; the rest were dropped (no memory update).

=== SUCCESS rules (beat the best baseline) ===

- obj=1230.0 | feas=0.92 | mean_tard=180.0 | expr: $0.0 - \max(\text{due}, \text{now} + \text{proc}) / \max(\text{penalty}, 0.001)$

=== FAILURE rules (lost to the best baseline) ===

- obj=1320.0 | feas=0.93 | mean_tard=195.0 | expr: $(\text{penalty} / \max(\text{proc}, 0.001)) * \exp(0.0 - \text{slack} / 3.0 / \text{proc})$

Your task: extract TRANSFERABLE lessons. For each lesson worth remembering for the NEXT iteration, output one block in EXACTLY this format (no preamble, no markdown, no commentary outside the blocks):

LESSON:

type: success | failure | strategy

title: <short imperative phrase>

description: <when it applies (which scenario, which condition)>

content: <the specific pattern, formula, weight choice, or pitfall>

perf_delta: <signed percent vs the best fixed baseline; +X means improvement>

END

Output 2–4 lessons. Prefer patterns about WHICH VARIABLES mattered, which weights were too aggressive, which conditional thresholds worked. Avoid restating rule expressions verbatim – extract the principle.

6. 반성가가 답해야 하는 형식

LESSON:

type: success

title: Penalty 가중치를 분모에 두는 구조가 강함

description: S5 복합 충격에서 납기/처리시간을 함께 보는 상황

content: $\max(\text{due}, \text{now} + \text{proc})$ 를 penalty로 나누는 WMDD-식 구조가 안정적

perf_delta: +12.5

END

LESSON:

type: failure

title: ATC 지수항이 단독으로는 충분치 않음

description: S5에서 supply-chain 변수를 무시했을 때

content: $\exp(-\text{slack} / (k * \text{proc}))$ 만 쓰면 mat_risk를 못 반영해 -7% 손해

perf_delta: -7.0

END

LESSON:

type: strategy

title: 복합 충격에선 due-aware + penalty-aware 결합 권장

description: disruption_level이 0.4 이상인 모든 시나리오

content: WMDD의 안정성 + ATC의 긴급도 인지를 가중치로 섞기

perf_delta: 0.0

END

시스템은 LESSON: ... END 블록을 모두 추출해서 메모리 뱅크에 저장합니다.

7. 사용하는 OpenAI API 설정

```
client.chat.completions.create(  
    model="gpt-4o-mini",          # 또는 gpt-4o, gpt-4.1  
    messages=[  
        {"role": "system", "content": <위 시스템 프롬프트>},  
        {"role": "user", "content": <위 본문 프롬프트>},  
    ],  
    max_completion_tokens=512,    # 생성자  
    # max_completion_tokens=1024, # 반성가 (LESSON 여러 개)  
)
```

중요한 점: - temperature는 기본값(1.0) 사용 — 다양성 확보 - max_completion_tokens=512로 짧게 끊음 (수식 한 줄이라 충분) - 캐싱·재시도 없음 — 매 호출 새로 생성

8. 비용 추정 (gpt-4o-mini 기준)

항목	토큰	비용 (한 번 호출)
입력 (생성자)	≈ 2,000 토큰 × \$0.15/Mtok	\$0.0003 (≈0.4원)
출력 (생성자)	≈ 200 토큰 × \$0.60/Mtok	\$0.00012 (≈0.16원)
입력 (반성가)	≈ 2,500 토큰	\$0.0004 (≈0.5원)
출력 (반성가)	≈ 600 토큰	\$0.00036 (≈0.5원)
한 번 사이클 (7 호출)		≈ 2~3원
풀 paper 실험 (~250 호출)		≈ 70원

9. 안전장치 — 잘못된 답이 와도 시스템이 안 망함

GPT가 가끔 잘못된 변수 이름이나 함수를 쓰면:

1. **alias 안전망:** urgency라고 써도 urgent로 받아줌 (7가지 흔한 오타 alias 등록됨)
2. **시뮬레이터 거부:** 정말로 잘못된 수식이면 시뮬레이터가 “이 규칙 불가능”이라고 에러
3. **루프 처리:** 에러난 규칙은 점수 1e9로 처리 → 자동 도태
4. **clamp 안전화:** clamp(x, 5, 1)처럼 거꾸로 된 경계도 자동 swap

이 덕분에 GPT가 한 번씩 실수해도 진화는 계속됩니다.

10. 한 줄 요약

우리 시스템 = “GPT에게 12개 비교 대상과 30개 변수와 6개 함수를 알려주고, 짧은 우선순위 수식 한 줄을 짜오라고 부탁한 뒤, 그 결과를 채점해서 다시 GPT에게 회고 노트를 받는 장치”입니다.

자세한 데이터 흐름은 [PIPELINE.md](#) 참고.